📁 AGENTS.md y skills enseña a tu agente a trabajar

Codificar con agentes · Parte 3 · Octubre 2026

👤 Sergi Garcia Barea

CC BY-SA 4.0

🧠 El agente no lee la mente (ni la memoria)

🔁 Sin contexto, en cada sesión

- Repite las mismas preguntas: "¿qué testing usas?"
- Ignora tus convenciones de nomenclatura
- Refactoriza donde no debía
- Hace commits con mensajes genéricos

📁 Con contexto, en cada sesión

- Ya sabe el stack, la estructura y los tests
- Respeta tu estilo desde el primer prompt
- Sabe qué no tocar y qué avisar
- Sigue el flujo del equipo, no su instinto

🎯 La solución no es repetirte cada día: es **poner ese conocimiento en ficheros que el agente lee**.

🔊

📁 AGENTS.md: el manual del proyecto

Un archivo **AGENTS.md** en la raíz del repo. Los principales agentes lo leen automáticamente al arrancar (OpenCode, Claude Code, Codex, Antigravity…).

🧩 Qué va dentro

Stack y comandos · estructura de carpetas · convenciones de código · cómo se testea · cómo se commitean los cambios · qué NO tocar

🌍 Fichero estándar

No es una invención de un agente: es un estándar de facto que varios agentes comparten. Va al repo y viaja con el código.

🏠 Dónde vive

Uno por proyecto en la raíz; se puede repetir en subcarpetas (p. ej. `backend/AGENTS.md` con reglas solo del backend).

👥 Para humanos también

Un onboarder nuevo (humano o agente) puede leerlo. Documenta decisiones del equipo en Markdown.

💡 Si explicarías esto a un compañero el primer día… eso va en AGENTS.md.

🔊

🧱 Estructura recomendada

📝 ESQUELETO DE AGENTS.MD

```
# Proyecto: cita-te

## Qué es
Gestor de citas para clínicas. Single-user demo.

## Comandos
- npm run dev      → arranque local
- npm run test     → tests (siempre antes de commit)
- npx prisma studio → ver la BD en local

## Convenciones
- TypeScript estricto; sin any
- Componentes en PascalCase, hooks en camelCase
- Commits: Conventional Commits, en español
- Comentarios del código en español

## No tocar
- src/lib/legacy/ (código antiguo, solo parches)
- Nunca subir ficheros .env ni secretos
```

🔊

Corto y accionable: el agente lo lee en cada sesión. Un AGENTS.md de 200 líneas nadie lo lee (ni humano ni IA).

🛠️ Qué escribir de verdad

✅ Reglas accionables
"Usa React Query para las peticiones, no `useEffect` a mano" > "sé consistente".

✅ Los comandos exactos
El agente ejecuta *tu* forma de testear, compilar y desplegar sin preguntar.

✅ Un ejemplo corto
Un fragmento de "cómo se hace aquí" vale más que diez párrafos.

✅ Lo que suele fallar y cómo se resuelve
"Si importas la BD, usa docker compose, no la del sistema."

❌ Lo que NO va
Toda la historia del proyecto, requisitos que ya están en tickets, objetivos que no afectan al código.

🔁 Mantenimiento: cuando el agente haga algo que no le gustó, **añade la regla** que faltaba en vez de corregirlo solo.

🔊

🎨 Skills: manual por tipo de tarea

AGENTS.md describe el proyecto. Un **skill** enseña a hacer bien un tipo concreto de tarea, y el agente lo consulta cuando lo necesita.

📄 Skill: crud-spring

"Cuando te pidan un CRUD: usa este esqueleto de controller/service/repository, estas validaciones y este patrón de tests."

🧪 Skill: pytest-rapido

"Estructura de factories, fixtures, nombres AAA y cobertura mínima del módulo nuevo."

📱 Skill: pantallas-android

"Listas con RecyclerView + DiffUtil, strings en resources, patrón ViewModel."

📄 Skill: doc-proyecto

"Cuando pidan README o memoria: esta plantilla de secciones, ejemplos de uso, cómo ejecutar."

🐳 Skill: dockerizar-servicio

"multi-stage build, imagen base pequeña, healthcheck y docker-compose con volúmenes."

🔁 Skills instalables

Hay skills de la comunidad (npx skills find / add) y puedes escribir los tuyos en `.agents/skills/`.

💡 Un skill = Markdown con instrucciones + ejemplos. Se lo enseñas una vez y lo aplica siempre.

🛒 Instalar skills de la comunidad

🔍 Buscar

```
npx skills find "react testing"
```

📜 Ver qué hace

```
npx skills add owner/repo --list
```

Lee el skill antes de instalar: describe su contenido y qué tocará.

⬇️ Instalar en el proyecto

```
npx skills add owner/repo \\
   --skill code-review --yes
```

📥 Usarlo

"Carga el skill code-review y revisa mi PR." El skill viaja en `.agents/skills/` con el repo.

⚠️ Los de la comunidad no están verificados: revísalos antes de instalar, igual que cualquier dependencia.

🔊

✍️ Tu propio skill

📝 .AGENTS/SKILLS/HOMEPAGE-WEB/SKILL.MD

```
---
name: homepage-web
description: Crea páginas web de práctica con JS puro y sin build
---

# Homepage web  (cuando te pidan una web educativa de práctica)

Escribe HTML+CSS+JS en ficheros separados, sin frameworks
ni paso de build: que funcione abriendo index.html.

## Reglas
- Móvil primero; probar también en pantallas anchas
- Datos concretos de muestra dentro de un array JS
- Español, ejemplo accesible (contraste, focus visibles)
- Sin librerías: DOM nativo, eventos y fetch solo
  si la web consume datos externos

## Verificar antes de entregar
- Sin errores en consola al cargar
- Probarlo con el navegador (con MCP de navegador si lo tienes)
```

🧭 AGENTS.md vs Skill: cuál usar

	AGENTS.md	Skill
Ámbito	Todo el proyecto, siempre activo	Una tarea, se consulta al hacerla
Contenido	Comandos, convenciones, "no tocar"	Recetas, plantillas, ejemplos completos
Ejemplo	"Tests primero al commitear"	"Cómo se escribe un endpoint REST aquí"
Tamaño	Lo mínimo (leer cada sesión)	Lo detallado (solo cuando aplica)
Reutilizable	Por repositorio	Copiado entre proyectos

🎯 Regla rápida: si es sobre **este repositorio**, AGENTS.md; si es sobre **cómo se hace X**, skill.

🧪 Ejemplo guiado: el skill en acción

💬 PROMPT

```
"Carga el skill crud-spring.

Añade el CRUD de 'almacenes':
entidad con nombre, ubicación
y capacidad. Endpoints REST,
validación con Bean Validation
y tests con MockMvc.

Sigue el esqueleto del skill,
no lo improvises."
```

OpenCode · Skill: crud-spring

💡 LO QUE CAMBIA CON SKILL

- ✅ El esqueleto de clases ya está decidido: menos decisiones sueltas
- ✅ Nombres y paquetes idénticos al resto del proyecto
- ✅ Los tests siguen el patrón acordado
- ✅ El resultado es comparable entre personas y sesiones
- ✅ La entregar es casi "pegar y adaptar"

🔊

🧠 Contexto extra que el agente lee

📖 README.md
Primera lectura de cualquier agente: qué es, cómo arranca, cómo se prueba.

🚧 TODO bien escrito
"TODO: validar límite de 50 archivos" — el agente los encuentra y te los propone si le pides la lista.

🗂️ Docs en el repo
Carpeta `docs/` con decisiones y diagramas. El agente la consulta en modo plan.

🗃️ Historia de Git
Buenos mensajes = el agente entiende qué se hizo y por qué sin preguntar.

🔗 Referencias concretas
"Guíate por el patrón de `src/modulos/clientes/`" — señalar un buen ejemplo existente funciona muy bien.

💡 Todo lo que escribiste para un humano también sirve para el copiloto. Buena documentación = mejor IA.

🔊

🌱 Empezar con el AGENTS.md live

No lo escribas de teoría: déjalo que lo construya con tu ayuda en la primera semana.

🚀 Día 1

```
"Genera un primer AGENTS.md del
proyecto con lo que has aprendido
al explorar: stack, comandos y
estructura. Que quede corto."
```

🧩 Cada semana

```
"Añade a AGENTS.md la regla de
que los formularios validan con
zod y no con su propio código.
Y borra la regla repetida."
```

🎯 El AGENTS.md es código: se versiona, se revisa en PR y se discute. Cuando alguien corrija al agente por algo que ya sabía el equipo… regla nueva al fichero.

🔊

✨ En esta parte 3

📁

AGENTS.md

El manual del proyecto, leído en cada sesión · corto · accionable · versionado en Git

🎨

Skills

Recetas por tipo de tarea · instalables de la comunidad · propios en `.agents/skills/`

🧭 **Uno u otro:** el repo → AGENTS.md · cómo se hace X → skill

🔁 **Vive:** se actualiza cada vez que aprendes algo del agente

Siguiente: **Ejemplos de desarrollo** — código, configuraciones y scripts reales, paso a paso. 💻

CC BY-SA 4.0 · Sergi Garcia Barea · Octubre 2026

🔊